

RL Lane — Handoff

Version 2 · valid as of 2026-09-05. Supersedes `RL_LANE_HANDOFF.pdf` (2026-09-01) and `RL_LANE_HANDOFF.html` (2026-09-03), both of which predate D29/D31 and D43–D46 and still print a six-rung ladder. **Delete them; this file is the source.**

Audience: the 2–3 people building rung 7. **Owner of this file:** Aamir. **Regenerate the PDF** with `python -m scripts.md2pdf docs/project/RL_LANE_HANDOFF.md`.

Four things in this document are marked [] PENDING. They are waiting on a scheduler-design brainstorm that is still running, and on two decisions Aamir owes. **None of them blocks Day 1.** Everything not marked [] is frozen, measured, or recorded, and will not move under you.

0. Read this first: the one paragraph that matters

The environment is **built, validated and frozen**. Seven non-learning schedulers have already been measured on it over 1,539 episodes. Your job is **rung 7** — a learned scheduler — and the bar is not the obvious one. Round-robin is beaten by almost everything. **The bar is rung 5, a one-line heuristic**, and the specific target is: *hold rung 5's 3.20 s censored intercept time while pushing its 0.110 interception ratio upward*. You do not get to pick your own metrics, your own scenarios, or your own seeds — `EVALUATION.md` fixes all three, and `rfenv/compare.py` already implements them.

1. Where the project stands

Layer	File	State
L0 scenario	<code>rfenv/scenario.py</code>	built, frozen
L1 truth grid	<code>rfenv/truth.py</code>	built, frozen
L2 receiver	<code>rfenv/receiver.py</code>	built, frozen
L3 agent interface	<code>rfenv/env.py</code>	built, frozen
artefacts / metrics	<code>rfenv/metrics.py</code> , <code>rfenv/render.py</code>	built
validation gates	<code>rfenv/validate.py</code>	ran 2026-09-04; 3 PASS, gate 1 MEASURED
baseline ladder	<code>rfenv/baselines.py</code> , <code>rfenv/compare.py</code>	ran 2026-09-04; rungs 1–6a
rung 7 — RL	does not exist	yours

229 tests pass (`.venv/bin/python -m pytest tests -q`, re-run 2026-09-05).

The environment is frozen (D42). `rfenv/constants.py` is closed — band geometry, slot clock, dwell schedule, `N`, `σ`, `γ`, `PD_POPULATION` — and `tests/test_freeze.py` pins every value as a literal plus a SHA-256 digest over the whole list. **If you edit `constants.py`, the suite goes red, and by D25 the environment must be re-validated from gate 1 and every baseline re-run.** Do not edit it. If you believe there is a genuine bug in it, that is a conversation with Aamir, not a commit.

Not frozen, deliberately, and therefore yours to move: the reward (three candidates already written), the observation vector's *extensions* (D30), and the per-episode scenario draw.

2. The five constraints you must not get wrong

These are not style preferences. Each is a recorded decision, and each has already caused a withdrawn number in this project.

1. The observation ships; the reward does not (D29). Training is offline and the policy is frozen before deployment, so your **reward may read truth** — `Z`, per-emitter levels, `first_e`, anything in `info`. Your **observation may not**. It is built only from the agent's own scan history, because that is all a fielded receiver has. `baselines.guarded()` enforces this by stripping `info` down to `OBSERVABLE_INFO` before a policy sees it; **wrap your policy in it too**, so a `KeyError` catches a leak instead of a suspiciously good score doing it.

2. An episode is 600 slots, not 600 steps. Seven of the 36 bands carry a 100 ms dwell and consume two slots, so an episode runs **300 to 600 step()** calls while always covering exactly 30 s. Airtime is the only currency. Consequences for you: episode lengths vary, so anything you compute per-step (advantage normalisation, entropy schedules, "average reward per step") is measuring something different from what the metrics measure. Rewards are **per slot** and a dwell's reward is the **sum** of its slots' (D31), so a wide band pays up to +2 — that is what keeps reward per unit time equal across band widths, and it is why the naive "+1 per dwell" is wrong.

3. Never report a single metric. Interception ratio, censored intercept time and emitter coverage go together, always (D14, `EVALUATION.md` §4). A camper scores 0.209 ratio and looks strong until you see 9.67 s and 0.497 coverage beside it. **A reward is not a metric** — average reward is comparable only within one reward family and never ranks across families (D7).

4. Train on the 47 train configs only. The 45 held-out test pairs are touched **once**, at the end, after the system is frozen (D8, `EVALUATION.md` §7 step 6). Not for a sanity check, not for "just one number". Every use gets recorded.

5. Never train or compare on a scan replay (D36). A scan recording holds only the pulses Turing's own sweeping receiver was tuned to, so a grid built from one hands any sweeping scheduler its answer — measured, interception ratio 0.9999 and censored intercept time 0.00 s. `rfenv/compare.py` raises `ScanReplayRefused` rather than trusting you. Train on **sampled scenarios**; evaluate on **stare replays + sampled scenarios**.

3. Your target, quantified

From `EVALUATION.md` §5, measured 2026-09-04, 1,539 episodes at $\gamma = -111$ dB, reward `hit_z`:

#	scheduler	interception ratio	censored intercept time (s)	coverage	beats round-robin on both
1	random	0.0669	4.16	0.858	42.1%

#	scheduler	interception ratio	censored intercept time (s)	coverage	beats round-robin on both
2	round-robin (equal airtime)	0.0605	4.18	0.865	—
3	Turing reference sweep	0.0805	3.74	0.864	51.5%
4	greedy camper (observation-fed)	0.2088	9.67	0.497	1.8%
5	recency / activity — the bar	0.1104	3.20	0.897	70.2%
6a	Apfeld: Active RFs	0.1320	4.32	0.860	53.2%
6	Apfeld adaptive (no tracking)	0.2455	14.86	0.367	4.1%
—	camper, truth-fed (<i>reference line</i>)	0.5680	15.71	0.296	7.0%
—	pulse-capture oracle (<i>reference line</i>)	0.6579	8.01	0.691	19.3%

Success ladder, in order:

- **Minimum** — beat round-robin on both metrics on >70.2% of paired episodes. Rung 5 already does this with four lines of code. Failing to reach it means something is wrong, not that RL is hard.
- **The real result** — Pareto-dominate rung 5: ratio > 0.1104 *and* intercept time < 3.20 s, paired per episode.
- **The claim worth making** — beat rung 6 (Apfeld) on ratio *without* its 14.86 s intercept-time collapse. Stated precisely: our adaptation of Apfeld's no-tracking variant on a binary-detection receiver, at the parameters in `baseline.ApfeldParams` (D44).

The two reference lines are not competitors and never appear unlabelled. `oracle_pulse` is a ceiling for interception ratio **only** — it *loses* censored intercept time to plain round-robin on 80.7% of episodes.

4. What to learn, and how long to spend

Nobody on this lane needs to become an RL researcher in five days. You need enough to make one algorithm work and to know when it is lying to you.

Domain — everyone, ~2 hours, on Day 0

You are new to RF/EW; that is expected. The whole domain, for our purposes, is:

- **Band, dwell, sweep.** The spectrum is split into 36 bands. The receiver can listen to exactly one at a time, for that band's fixed dwell (50 or 100 ms). A full pass over all bands is a sweep.
- **Why this is hard.** Emitters are rotating radars — they illuminate you only when their beam sweeps past. So you are trying to be tuned to the right band *at the right moment*, with no prior knowledge of who is out there (D20: every episode starts cold — the PS's "*absence of prior reliable intelligence*" made literal).
- **Interception ratio** = of all the moments an emitter was illuminating us, what fraction were we listening for. **Censored intercept time** = how long after an emitter became detectable did we first catch it, counting a 30 s miss as 30 s.
- **The right framing, and say it in the PPT too:** this is a **restless multi-armed bandit** — 36 arms, each arm's payoff evolving whether or not you pull it, under partial observability. That one sentence tells you what literature to read.

Read, in this order: SIH26055_PROBLEM_STATEMENT.md (10 min) → EVALUATION.md §4 and §5 (30 min) → the module docstring at the top of [env.py](#) (15 min, it is written for you) → DECISIONS.md D28, D29, D31, D34, D46 (45 min).

RL — the training owner, ~4 hours

- MDP, episode, return, discounting. Why $\gamma_{\text{RL}} < 1$ matters here: a 30 s episode with up to 600 steps is long enough that $\gamma_{\text{RL}} = 0.99$ has a ~100-step horizon, which is shorter than an episode.
- **Policy gradient vs value-based**, at the level of "PPO is a policy-gradient method with a clipped update that stops the policy moving too far in one step". You do not need the proof.
- The Gymnasium API surface: `reset()` → `(obs, info)`, `step(a)` → `(obs, r, terminated, truncated, info)`. Our env already implements it — read `env.py`, it is your tutorial.
- **Action masking is not needed.** All 36 bands are legal at every step.
- Vectorised environments, and why they change what a "step" means in your logs.

RL — the evaluation owner, ~2 hours

- Paired comparison: same scenario, same seed, same truth grid, per episode. A mean can clear a mean while losing most episodes — that is exactly why the ladder's last column exists.
- Seeds and spread. EVALUATION.md §7 forbids single-run numbers.
- **Overfitting to 47 scenarios.** Train on `pool=` (fresh sampled scenario every reset), never on a fixed scenario, or you are memorising (D25, D32).

Skip entirely

Model-based RL, offline RL, multi-agent, transformers, curriculum learning, hyperparameter search frameworks. Five days.

5. Work split

Three roles. The reason for the split is the second one: the person who trains the agent must not be the only person who evaluates it. That is the single most common way a hackathon ML result turns out to be wrong, and our whole credibility story (§8) collapses if it happens.

Role	Owns	Deliverable
R1 — Training	the algorithm, the training loop, hyperparameters, checkpoints, the <code>RLScheduler</code> policy class	a checkpoint that loads and plays an episode

Role	Owens	Deliverable
R2 — Evaluation & integration	run 7 in <code>baselines.LADDER</code> , the comparison runs, seeds, the Pareto plot, the held-out run	the §3 table with a row 7 in it
R3 — Reward & observation	the three reward candidates, D29's selection rule, the D30 experiment, ablations	a Pareto front over rewards, and D30 answered

If you are two people: R1 and R3 merge (they share the training harness anyway). **R2 stays separate — do not merge it into R1.**

[] **PENDING — names against R1 / R2 / R3.** Owner: Aamir. Fill in before Day 1 starts.

6. The task list

Prerequisites for everyone, before any of this: `.venv` active, `pytest tests -q` green, and `python -m rfenv.compare --seeds 1 --sampled 2 --out runs/scratch` run once so you have seen the artefacts the pipeline produces.

Day 1 — Get an agent to move, and get it measured

#	Task	Owner	Done when
1.1	Add <code>stable-baselines3>=2.3</code> and torch to <code>requirements.txt</code> , install	R1	<code>import stable_baselines3</code> works
1.2	<code>rfenv/rl.py</code> — <code>make_train_env()</code> returning <code>ScanEnv(pool=..., reward=...)</code> , <code>check_env</code> clean	R1	Gymnasium's <code>check_env</code> passes
1.3	Train PPO for a token number of steps on <code>hit_z</code> . Do not tune anything.	R1	a <code>.zip</code> checkpoint exists
1.4	<code>RLScheduler(checkpoint)</code> — a callable (<code>obs, info</code>) -> <code>int</code> , wrapped in <code>baselines.guarded()</code>	R2	it plays an episode without a <code>KeyError</code>

#	Task	Owner	Done when
1.5	Add <code>Rung("rl", "7", ...)</code> to <code>baselines.LADDER</code> ; <code>compare.py</code> picks it up with no other change	R2	--rungs rl, recency, round_robin runs
1.6	Read <code>DECISIONS.md</code> D28/D29/D31/D34/D46; write the D29 selection-rule proposal for Aamir	R3	a one-paragraph rule, in writing

Day 1 ends with a bad agent that is fully measured. That is the point. An untuned PPO scoring below random, appearing correctly in the §3 table, is worth more on Day 1 than a good agent you cannot score.

Day 2 — Make it actually learn

#	Task	Owner	Done when
2.1	Vectorised training (8–16 envs), longer run, learning curve logged	R1	reward curve rises and flattens
2.2	Sweep the three things that matter and nothing else: <code>γ_{RL}</code> , entropy coefficient, <code>n_steps</code>	R1	a table of ~6 runs
2.3	Evaluation harness: N seeds × the §5 scenario set, paired against <code>recency</code>	R2	a paired-win % for rung 7
2.4	Train one agent per reward candidate (<code>hit_z</code> , <code>hit_y</code> , <code>first_intercept</code>)	R3	three checkpoints
2.5	Confirm the observation is not leaking: assert the policy's input has no truth-side component	R2	a test in <code>tests/</code>

Gate: does rung 7 beat round-robin on both metrics on >50% of paired episodes? If not, stop tuning and go back to 2.1 — something is structurally wrong, not under-trained.

Day 3 — Beat the bar, and find out what is carrying the gain

#	Task	Owner	Done when
3.1	Longest training run you can afford; checkpoint often	R1	best checkpoint identified on <i>train</i> scenarios

#	Task	Owner	Done when
3.2	Full ladder run with rung 7 in it, <code>--seeds 3</code> <code>--sampled 10</code> <code>--figures</code>	R2	<code>runs/baselines/comparison.md</code> has a row 7
3.3	Pareto front over the three rewards; apply D29's selection rule; record the choice and why	R3	one reward chosen, in <code>DECISIONS.md</code>
3.4	D30 experiment — does adding AoA / PulseWidth to the observation help? Train with and without	R3	a paired comparison, and D30 closed either way
3.5	Ablation: rung 7 vs rung 5 vs rung 6a, one paragraph on <i>which component earns the gain</i>	R2	written

Day 4 — Freeze, then test once

#	Task	Owner	Done when
4.1	Freeze the final system: one checkpoint, one reward, one observation spec, committed	all	a commit tagged as the frozen system
4.2	The held-out run. 45 test pairs, once. Record the date, the command, the checkpoint hash	R2	numbers in <code>EVALUATION.md</code> , use recorded per D8
4.3	Regenerate all figures from the frozen system	R2	<code>pareto.png</code> , <code>timeline_*.png</code> , <code>discovery_*.png</code>
4.4	Write the rung-7 section of <code>EVALUATION.md</code> §5 and a D4x decision recording the run	R1+R2	committed

4.2 is irreversible. If the held-out set is touched before 4.1, the final evaluation means nothing and we cannot honestly claim it. Do not run it early "just to see".

Day 5–6 — Buffer, demo, and the edge

#	Task	Owner
5.1	A live demo path: load checkpoint → run one episode → render the timeline. Under 30 s	R1
5.2	Record a backup demo video locally. The playbook's most-cited failure is a live demo that didn't run	R1
5.3	Hand the PPT lane: final numbers, final figures, the one-sentence claim	R2
5.4	The edge (§9) — whichever candidate the numbers supported	all

7. The integration point, concretely

Rung 7 plugs in at exactly one place. Nothing in `env.py`, `metrics.py` or `compare.py` needs to change.

```
# rfenv/rl.py
class RLScheduler:
    """A trained policy as a `(obs, info) -> band` callable, per metrics.run_episode."""
    def __init__(self, model):
        self.model = model
    def __call__(self, obs, info) -> int:
        action, _ = self.model.predict(obs, deterministic=True)
        return int(action)

# rfenv/baselines.py – one entry appended to LADDER
Rung("rl", "7", "RL scheduler",
    "Ours. Trained on sampled scenarios; sees only the D34 observation.",
    lambda rng, grid: RLScheduler(load_checkpoint()),
```

`baselines.make()` wraps every deployable rung in `guarded()` automatically, so your policy is held to the same information constraint as every other rung — which is the entire reason the comparison means anything.

Training env, not evaluation env:

```
from rfenv.env import ScanEnv
from rfenv.scenario import EmitterPool

env = ScanEnv(pool=EmitterPool.from_train(), reward="hit_z") # pool, never scenario
```

`pool=` draws a fresh scenario every `reset()`, so no episode repeats and there is nothing to memorise (D25, D32). A fixed `scenario=` is for the validation gates, not for training.

8. Evaluation is the deliverable

Read this section twice. **A trained agent with a number nobody can reproduce is worth less than no agent at all** — this repository exists because a previous attempt accumulated measured claims that could not be traced back to what produced them.

Rules, all from EVALUATION.md §7:

1. **Paired, per episode.** Same scenario, same seed, same truth grid. Report the win rate, not just the means.
2. **Multiple seeds, with spread.** A single run is not a result.
3. **State the operating point** — γ , P_{fa} — beside any scheduler table. And name the population P_d was averaged over: it is *not* data-independent (D33, D46).
4. **Distributions, not a grand mean.** Scenario difficulty spans 2 to 99 emitters.
5. **Never bare accuracy.** With sparse occupancy, "predict nothing" scores well and is useless.
6. **Label reference lines as reference lines** in every table they appear in.

What would falsify our result, and how we check it — have an answer ready for each, because a judge who knows RL will ask:

Objection	Our check
"You overfit to 47 scenarios"	trained on sampled scenarios (D25); held-out run at task 4.2
"Your agent saw the truth"	<code>guarded()</code> raises <code>KeyError</code> ; test at task 2.5
"You picked the metric that flattered you"	three metrics, always together, fixed before the run
"You tuned the threshold after seeing results"	gate criteria fixed in <code>validate.py::GATES</code> before the first run, asserted by a test (D39)
"Your baseline was weak"	round-robin is beaten by <i>random</i> ; the bar is rung 5 and Apfeld

9. The edge

An SIH deck needs one sentence a judge repeats afterwards. **We do not have it yet**, and the scheduler brainstorm may produce it. What follows is the *method* for finding one, and four candidates the measurements already support — offered so the brainstorm has something to push against, not as a decision.

How to find an edge, in this project

1. **An edge must be a number, not an adjective.** The differentiation strategy doc's own rule (§27): "*make numerical claims, not marketing claims.*" If a candidate edge cannot be written as a row in the §3 table or a line in a gate report, it is a slogan.
2. **Look where a measurement surprised us.** Surprises are where the insight is, because they are where the obvious model was wrong. This project has produced four: D36, D43, D45, D46.
3. **Two-sentence test.** Sentence one states it in language a judge with no RF background understands. Sentence two is the number that backs it. If sentence two doesn't exist yet, name the experiment that would produce it and put it on the Day 3 list.
4. **Kill criterion, agreed in advance.** If the supporting number is not measured by end of Day 3, the candidate is dropped from the deck. An edge you have to hedge is worse than one you don't claim.

Candidate A — the operator's dial (*recommended as the tagline*)

Claim: *No single scheduler is best, because the two things the problem statement asks for pull against each other. So we don't ship one — we ship a front, and the operator picks a point on it at mission time.*

Why it's true and already recorded: D28 puts the two headline metrics on **opposite sides** of the observability line — censored intercept time requires a declared detection $Y = 1$, interception ratio does not — so **no single reward is aligned with both** (EVALUATION.md §4). That is not a limitation we're admitting; it is a property of the problem that we can *demonstrate*, and it makes "which do you want today — find everything fast, or capture the most signal?" a real operational question rather than a slide.

What would make it land: a preference-conditioned policy — one network, an extra input weighting ratio against time — so the demo is a slider that visibly moves the agent's behaviour. That is genuinely more product than most SIH ML entries manage. **Cost:** ~half a day on top of a working agent. **Do not start it before Day 3.**

Candidate B — learning to see what the receiver cannot measure (*recommended as the technical claim*)

Claim: *The hard part isn't scheduling. It's that the quantity a good schedule depends on is invisible to the receiver — and we can measure exactly how invisible.*

The number: D46. A camper that reads the truth grid gets interception ratio **0.568**. The same camper strategy fed only what a real receiver observes gets **0.209**. That **0.359 gap is the price of not being able to see illumination density** — and D14's celebrated 57.4% camper was never reachable by any deployable scheduler. **Our claim becomes: rung 7 closes X% of that gap**, which is a quantity nobody else in the room will have.

Candidate C — we can prove our simulator is right (*credibility layer, not the tagline*)

Four validation gates, **criteria written into `validate.py`: :GATES before the first run and asserted by a test** so no threshold was chosen once the measurement was visible (D39). Plus D42, which records by fault injection what the gates *cannot* detect — including that no gate covers the ± 500 MHz band half-width. Almost no hackathon team validates its simulator at all, and none publishes its own blind spots.

Use this as the slide *under* the results, not as the headline: it wins the "do we believe you" question, which is where most ML pitches quietly lose.

Candidate D — what we tried that didn't work (*one slide, high value*)

- **D45:** Apfeld's period estimation *hurts* here — $1.9\times$ the ratio for $3.4\times$ the intercept time on 30 s binary episodes. We implemented the published method and it lost to its own ablation.
- **D36:** a scan replay hands any sweeping scheduler its answer (0.9999 ratio). We nearly benchmarked on it. Catching that is why our numbers are honest.
- **D43:** two of our own baselines turned out to be the same policy.
- **D44:** Apfeld's Algorithm 1 contradicts the paper's own prose; we implement the prose and say so.

Judges reward teams that can say what didn't work. Most decks cannot.

Do not claim

Superiority over fielded or classified military systems (differentiation doc §28), real-time hardware performance we haven't measured, or anything from `../SIHProto`.

[] **PENDING — the edge and the tagline.** Owner: Aamir + the brainstorm. Closed by: picking one of A–D (or something better), and naming the number that backs it. **Target: end of Day 2**, so the supporting experiment can still fit on Day 3.

10. Decisions you own

Decision	State	Who closes it	Blocks
D29 selection rule — what to do when reward candidates Pareto-dominate the baselines but not each other	[] owed	Aamir , before training	task 3.3
D30 — do AoA and PulseWidth enter the observation?	OPEN, explicitly " <i>owned by the RL lane</i> "	R3	task 3.4
RL algorithm	not recorded	R1 + brainstorm	task 1.3 — default to PPO and move on
The edge	[] pending	Aamir + brainstorm	§9

Record every decision you take in DECISIONS.md, including small ones, with the evidence and the reasoning. That file is how this project survives a session ending, and it is where the PPT lane gets its "what we tried" material.

11. Where things are

Need	Go to
what we were asked to build	docs/project/SIH26055_PROBLEM_STATEMENT.md
metrics, baselines, gates, protocol	docs/project/EVALUATION.md — the single authority; do not redefine a metric elsewhere
every decision and its evidence	docs/project/DECISIONS.md
the buildable spec	docs/project/ENVIRONMENT_SPEC.md
the agent interface, explained	the docstring at the top of rfenv/env.py
a claim in a PDF	<code>.venv/bin/python -m docsearch "your question" -k 8</code> — returns file.pdf:p.7 citations

Provenance rules apply to you (CLAUDE.md): every number traceable to a named source, a measurement you didn't run this session is not a fact, and a summary of a source is not a second source. If you quote a figure from this document without re-running it, say so.